Open-Source Book Ingest Tracker

Living doc tracking which freely-published books are flowing into the Fishbones catalog, and where each one is in the pipeline.

Last updated: 2026-04-28 Maintainer: matt + Claude agent runs

---

1. Selection criteria

A book makes the list if all of:

- **Freely published online** — CC license, GitHub source, or official-site free read. Paywalled / in-print-only books are out (we'd be retyping them).
- **Modular chapter structure** — chapters can become lessons without surgery.
- **Runnable code, or a clean place to attach a runnable language** — enough that we can derive exercises + drills, not just prose.
- **Won't duplicate the catalog** — adds depth (more JS, deeper Rust) or fills a gap (Lightning, OCaml).

Books that teach a *concept* without owning a single language (algorithms, OS, networking, blockchain protocols) get a `(with X)` suffix. The suffix names the runnable language **or** the popular client-library we anchor the exercise pane to, while the prose stays book-true. Where a domain has a de-facto JS/TS client library, we prefer it — Viem for Ethereum, bitcoinjs-lib for Bitcoin — so the exercise sandbox runs in the existing JS runtime without new compilers to wire up. Examples:

- *Mastering Bitcoin (with bitcoinjs-lib)* — Antonopoulos's prose, runnable TypeScript exercises built on **bitcoinjs-lib** (transaction crafting, script construction, HD wallets, signing) — the de-facto JS Bitcoin library.
- *Mastering Ethereum (with Viem)* — Antonopoulos & Wood's prose, runnable TypeScript exercises built on **Viem** — the modern Ethereum client (RPC, wallet ops, contract reads, ABI encoding).
- *Crafting Interpreters (with JavaScript)* — Nystrom's prose + Lox interpreter, ported to JS so it runs in our browser sandbox.
- *Algorithms — Jeff Erickson (with Python)* — concept-rich pseudocode in the prose, Python in the editor.
- *OSTEP (with C)* — OS concepts; the C code in the book's xv6 / mlfq examples becomes the exercise material.
- *Pro Git* — no suffix; shell + git only, and that's the point.

---

2. Wave 1 — start now

Highest signal-per-page, cleanest fit with the existing pipeline. These four go into ingest in this order; each spawns an agent that drafts a course shell, then runs through the same drill / quiz / cover / sync flow we used for the seed catalog.

#	Title	License	Language	Source	Est. lessons
1	**Eloquent JavaScript** (4th ed.)	CC BY-NC 3.0	JavaScript	eloquentjavascript.net + GitHub	~25-30
2	**Crafting Interpreters (with JavaScript)**	CC BY-NC-ND 4.0 (text), public-domain code	JavaScript (Lox port)	craftinginterpreters.com + GitHub	~30-35
3	**Programming Bitcoin (with Python)**	MIT (code)	Python	GitHub	~14
4	**Pro Git**	CC BY 3.0	shell / git	git-scm.com/book + GitHub	~10-12

3. Wave 2 — queue after Wave 1 lands

#	Title	License	Language	Source	Est. lessons
5	**Mastering Bitcoin (with bitcoinjs-lib)**	CC BY-SA 4.0	TypeScript	GitHub + bitcoinjs-lib	~12
6	**Mastering Ethereum (with Viem)**	CC BY-SA 4.0	TypeScript	GitHub + Viem	~14
7	**Mastering Lightning Network (with TypeScript)**	CC BY-SA 4.0	TypeScript	GitHub + bolt11, @noble/secp256k1	~16
8	**The Modern JavaScript Tutorial** (javascript.info)	CC BY-NC-SA 4.0	JavaScript	javascript.info	~120 (split into 3 sub-courses)
9	**You Don't Know JS Yet** (2nd ed.)	CC BY-NC-ND 4.0	JavaScript	GitHub	~30 across 6 short books
10	**Rust by Example**	Apache-2.0 / MIT	Rust	GitHub	~25

4. Wave 3 — adds new language coverage

These need runtime work (new compilers / interpreters wired into Fishbones). Defer until Wave 1+2 land and we have a clearer picture of authoring throughput.

#	Title	License	Language	Source	Notes
11	**The Async Book** (Rust)	Apache-2.0 / MIT	Rust	GitHub	TRPL companion
12	**The Rustonomicon**	Apache-2.0 / MIT	Rust	GitHub	Advanced/unsafe Rust
13	**Composing Programs (with Python)**	CC BY-SA 4.0	Python	composingprograms.com	UC Berkeley CS61A — Python flavoured SICP
14	**Open Data Structures (with Python)**	CC BY 2.5	Python	opendatastructures.org	Has Java/C++/Python — pick Python
15	**Algorithms — Jeff Erickson (with Python)**	CC BY 4.0	Python	jeffe.cs.illinois.edu/teaching/algorithms	Language-agnostic; Python in exercises
16	**Functional-Light JavaScript**	CC BY-NC-ND 4.0	JavaScript	GitHub	FP intro

5. Wave 4 — explicitly deferred

License-friendly and high quality, but each requires a meaningful runtime addition before the exercise pane works. Documented here so we don't lose them.

Title	Why deferred
Learn You a Haskell	No Haskell runtime in Fishbones.
Real World Haskell	Same.
Real World OCaml (with OCaml)	No OCaml runtime; could go via js_of_ocaml.
Operating Systems: Three Easy Pieces (with C)	OS concepts; C runs but most exercises rely on a Linux syscall surface we don't have.
xv6 book (with C)	Same — kernel-level work needs a sandbox.
Beej's Guide to Network Programming (with C)	Sockets — needs a network sandbox.

---

6. Per-book ingest status

Status legend: 🟢 not started → 🟡 drafting → 🔵 ingesting → 🟠 editing → ✅ shipped

#	Title	Status	Cover	Drills	Quizzes	Bundled?	Notes
1	Eloquent JavaScript	🟠	Pending Lovart	auto-derive next	drafted (3)	✓ `.fishbones` packed	31 lessons (11 reading + 17 exercise + 3 quiz) across 11 chapters; all 17 exercise solution pass their own Jest tests. Source ch00–12 covered; ch13–21 (DOM/Node) deferred.
2	Crafting Interpreters (with JavaScript)	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	30 lessons (10 reading + 18 exercise + 2 quiz) across 10 chapters; all 18 solutions pass first verification (122 individual test cases). Lox tree-walker fully ported Java→JS throug ch10 (classes + inheritance).
3	Programming Bitcoin (with Python)	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	23 lessons (9 reading + 12 exercise + 2 quiz) across 9 chapters; all 12 exercise solutions pass pytest first try. Source ch01-09 covered (finite fields → blocks). Sighash verified against book's own test vector.
4	Pro Git	🟠	Pending Lovart	n/a (no exercises in most lessons)	drafted (3)	✓ `.fishbones` packed	12 lessons (6 reading + 3 conceptual JS exercises + 3 quiz) across 6 chapters; all 3 exercise solutions pass (19/19 tests across the 3). Conceptual exercises: merge-base B 3-way merge, content-addressed store.
5	Mastering Bitcoin (with bitcoinjs-lib)	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	16 lessons (6 reading + 8 exercise + 2 quiz) across 6 chapters; all 8 exercise solutions p their own Jest tests. Source ch01-02, 04-08, 10, 12-14 covered. P2PKH derivation matc BIP32 test vectors.
6	Mastering Ethereum (with Viem)	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	17 lessons (7 reading + 8 exercise + 2 quiz) across 7 chapters; all 8 exercise solutions p their own Jest tests. Source ch1, 2, 4-7, 10, 12, 14-15 covered.
7	Mastering Lightning Network (with TypeScript)	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	18 lessons (7 reading + 9 exercise + 2 quiz) across 7 chapters; all 9 exercise solutions p their own Jest tests. Source ch01, 03, 06-08, 10-12, 15-17 covered; uses Node `crypto` DI pattern (no library mocks needed).
8	The Modern JavaScript Tutorial	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	31 lessons (9 reading + 20 exercise + 2 quiz) across 9 chapters; 20/20 exercises pass J Book 1 (Fundamentals) only — books 2 (Browser) and 3 (Frameworks) deferred. 8 exerc lifted from source `task/` + `solution/` pairs with attribution; 12 synthesized.
9	You Don't Know JS Yet	🟠	Pending Lovart	auto-derive	drafted (2)	✓ `.fishbones`	25 lessons (6 reading + 17 exercise + 2 quiz) across 6 chapters; 17/17 exercises pass Je Note: sync-async + es-next-beyond books are TODO in the 2nd-ed source repo — thos

#	Title	Status	Cover	Drills	Quizzes	Bundled?	Notes
				next		packed	chapters authored from canonical patterns.
10	Rust by Example	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	26 lessons (8 reading + 16 exercise + 2 quiz) across 8 chapters; 16/16 exercises pass `cargo test` (90 tests). Tests use bare `#[test]` to match `the-rust-programming language` runner pattern.
11	Async Book (Rust)	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	22 lessons (7 reading + 13 exercise + 2 quiz) across 7 chapters; 13/13 exercises pass `cargo test` (56 tests). Futures, pinning, streams, join!/select!.
12	Rustonomicon	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	20 lessons (8 reading + 10 exercise + 2 quiz) across 8 chapters; 10/10 exercises pass `cargo test` (54 tests). Every `unsafe` block carries a `// SAFETY:` soundness argument.
13	Composing Programs (with Python)	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	23 lessons (8 reading + 13 exercise + 2 quiz) across 8 chapters; 13/13 exercises pass pytest. CS61A canonical: larger/count_partitions/compose1/accumulate/rational ADT/Account+CheckingAccount/Church numerals/calc_eval.
14	Open Data Structures (with Python)	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	22 lessons (8 reading + 12 exercise + 2 quiz) across 8 chapters; 12/12 exercises pass pytest (96 tests). Solutions lifted from Morin's `python/ods/*.py` with attribution.
15	Algorithms — Erickson (with Python)	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	23 lessons (7 reading + 14 exercise + 2 quiz) across 7 chapters; 14/14 exercises pass pytest (103 tests). Recursion → divide-and-conquer → backtracking → DP → greedy → graphs → Dijkstra. Channels Erickson's voice (recursion fairy, exchange-argument proof
16	Functional-Light JavaScript	🟠	Pending Lovart	auto-derive next	drafted (2)	✓ `.fishbones` packed	23 lessons (9 reading + 12 exercise + 2 quiz) across 9 chapters; 12/12 exercises pass Je (69 tests). compose/pipe/curry/partial/deepFreeze/setIn/factorial/flatten/reduce/filterMapReducer/a — Simpson's FP-light primitives, hand-built.

7. Per-book ingest workflow

The pipeline used for the seed catalog (TRPL, Svelte 5, Crypto, etc.) — codified for re-use.

1. **Pick course id** — kebab-case slug, e.g. `eloquent-javascript`, `crafting-interpreters-js`, `programming-bitcoin`.
2. **Source format**:
   - GitHub markdown: clone, run a markdown→lesson splitter that respects chapter boundaries.
   - Live docs site: `crawl_docs_site` Tauri command.
   - PDF: `ingest_book` command.
3. **Spawn an authoring agent** with a prompt template (see `scripts/spawn-book-ingest.mjs`, to be created on first run). Agent's job:
   - Read source.
   - Emit a `course.json` matching `data/types.ts`.
   - Generate exercises with `starter` / `solution` / `tests` from the book's code blocks.
4. **Cover art**:
   - Per-book prompts already drafted in `docs/cover-prompts.md` (specimen-plate / armoury preamble).
   - Run via Lovart at 1024 × 1536, save to `cover-overrides/<course-id>.png`.
   - Style guide / palette: `docs/cover-art-style.md`.
5. **Drill authoring** — same agent flow we used for the seed catalog.
6. **Quiz expansion** — same agent flow.
7. **Difficulty tagging** — same agent flow.
8. **Cleanup pass** — same agent flow.
9. **Ship**:
   - Add to `scripts/extract-starter-courses.mjs` PACK_IDS.
   - Add `.fishbones` archive to `src-tauri/resources/bundled-packs/`.
   - Run `node scripts/extract-starter-courses.mjs && node scripts/sync-drills-to-local.mjs`.
   - Rebuild iOS sim + push.

8. Open questions

- **Multi-language books** — *The Modern JavaScript Tutorial* has a node section in book 2 ("server-side JavaScript"); should we tag it `bun` or `javascript`? Default: `javascript`.
- **Crafting Interpreters phasing** — ship as one course (both interpreters back-to-back) or two (`crafting-interpreters-treewalk` + `crafting-interpreters-bytecode`)? The book is ~400 pages each half; one-course feels right since the second half builds on the first.
- **Pro Git mediation** — git is a CLI, not a programming language. Our exercise sandbox can run shell, but `git init` / `git commit` need a writable filesystem context. Plan: ingest as quizzes + readings (fewer exercises) so we don't rebuild the sandbox just for git.

- **Licensing on the cover** — CC BY-NC-ND books restrict derivatives; our drill cards count as derivatives of the prose. Plan: keep drills strictly tied to the book's example code (which is permissively licensed in every Wave-1 entry) and treat prose as "for reading" (untouched).

---

9. Changelog

- **2026-04-28** — Tracker created. Wave 1 selected (Eloquent JS, Crafting Interpreters, Programming Bitcoin, Pro Git). Wave 2-3 queued. All 16 cover prompts drafted in `docs/cover-prompts.md` using the specimen-plate / armoury preamble (Bitcoin + Ethereum prompts ported from earlier chat). PDF version of this tracker generated alongside (`docs/openbook-ingest-tracker.pdf`).
- **2026-04-28** *(later)* — Re-anchored *Mastering Bitcoin* and *Mastering Ethereum* exercise spines to popular JS/TS client libraries: **bitcoinjs-lib** (Bitcoin) and **Viem** (Ethereum), instead of Python / Solidity. Both run in the existing browser sandbox — no new compiler wiring. Convention added to §1: where a domain has a de-facto JS/TS client lib, we prefer it.
- **2026-04-28** *(later still)* — All 15 books in Waves 1–3 have landed and bundled. **15 books · 369 lessons · 201 verified-passing exercises · 485KB compressed.** Sync-latest-courses button shipped (Settings → Data → "Sync now"). Wave 4 (Haskell, OCaml, OS-internals, network-programming) remains explicitly deferred pending new runtime work. Mastering Lightning re-anchored from Python to TypeScript (bolt11 + @noble/secp256k1) for visual + library consistency with Mastering Bitcoin/Ethereum.